

Où est mon temps ?

Application web pédagogique de gestion intelligente du temps — mesurer, comprendre, analyser, planifier, améliorer. Présentation technique pour étudiants en développement web et leur professeur.



Concept — en une phrase

Créer et prioriser des tâches, suivre le temps passé sur chaque activité, analyser les habitudes et proposer un planning optimisé pour le lendemain.

Mesurer

Chronométrer sessions de travail (TIME_SESSION).

Comprendre

Identifier tâches terminées et reportées.

Analyser

Extraire périodes de productivité et activités chronophages.

Planifier

Proposer un ordre et des créneaux pour demain.

Améliorer

Itérer l'algorithme en fonction des données utilisateur.

Architecture générale

Schéma simple : UTILISATEUR → FRONTEND (React + Tailwind) → API HTTP/JSON → BACKEND (Node.js) → DATABASE (PostgreSQL).

Rôles

- Frontend : UI, saisie des tâches, timer, visualisation.
- Backend : logique métier, API, authentification.
- Database : stockage persistant (tâches, sessions, objectifs).
- API : contrat JSON entre frontend et backend.

Arborescence du frontend

Structure principale et rôle de chaque dossier :

- `src/components` : composants réutilisables (Navbar, Sidebar, TaskCard, Timer, Statistics, DailyPlan).
- `src/pages` : vues (Login, Dashboard, Tasks, History, Profile).
- `src/services` : `api.js` — fonctions fetch pour communiquer avec l'API.
- `App.jsx` / `main.jsx` : point d'entrée et routage.
- `package.json` / `tailwind.config.js` : dépendances et configuration visuelle.

Maquette : Interface principale

Tableau de bord avec métriques (temps total, nombre de tâches, productivité), liste de tâches du jour, et chronomètre d'activité en cours. Priorités colorées : rouge=élevée, orange=moyenne, vert=terminée.



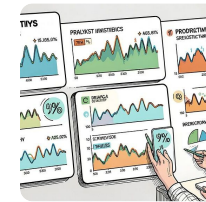
Liste de tâches

Ajout, toggle, priorisation, estimation temps.



Chronomètre

Start/Pause, sessions liées à une tâche



Statistiques

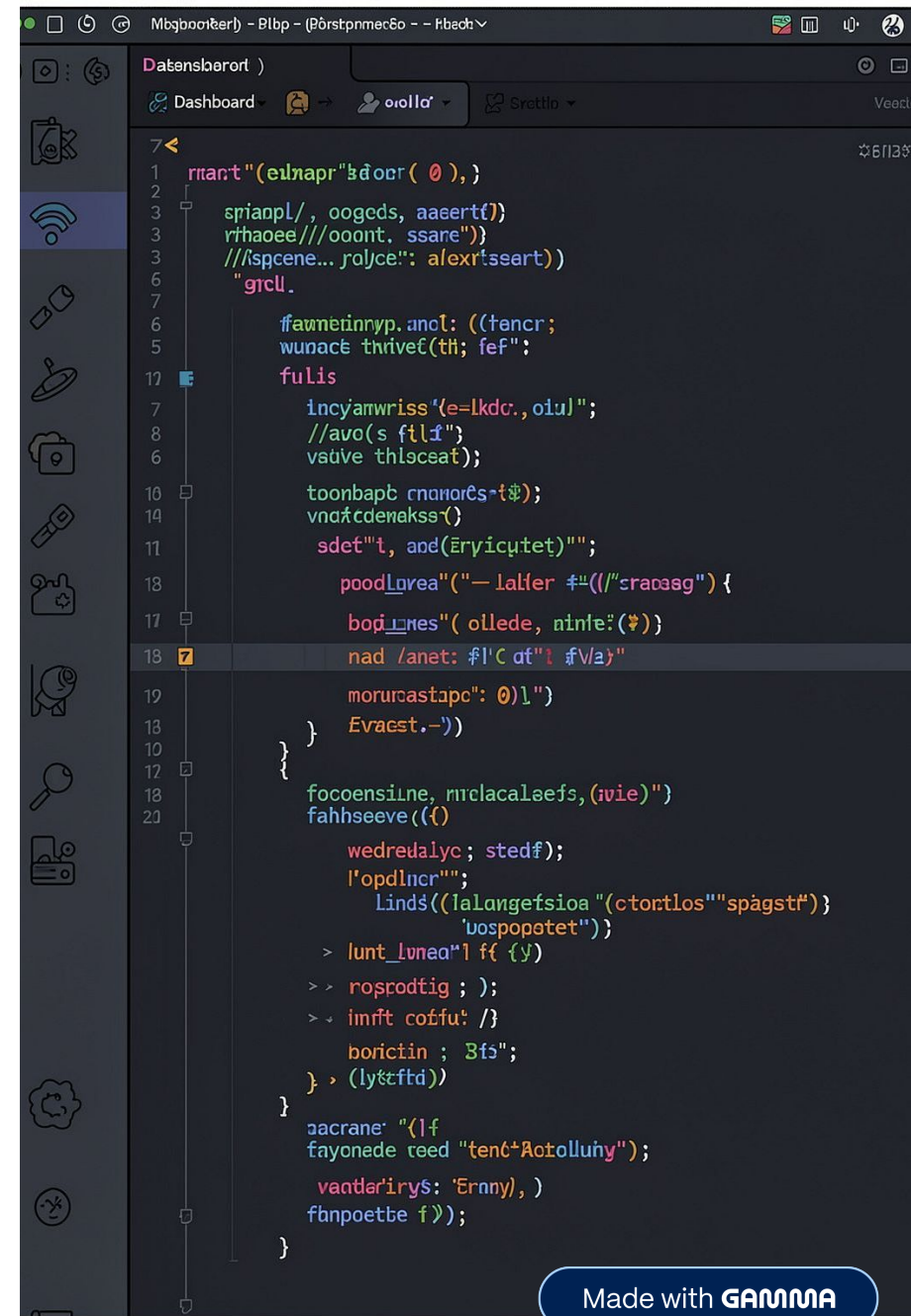
Périodes productives, tâches longues, habitudes.

Composant principal — code (extrait)

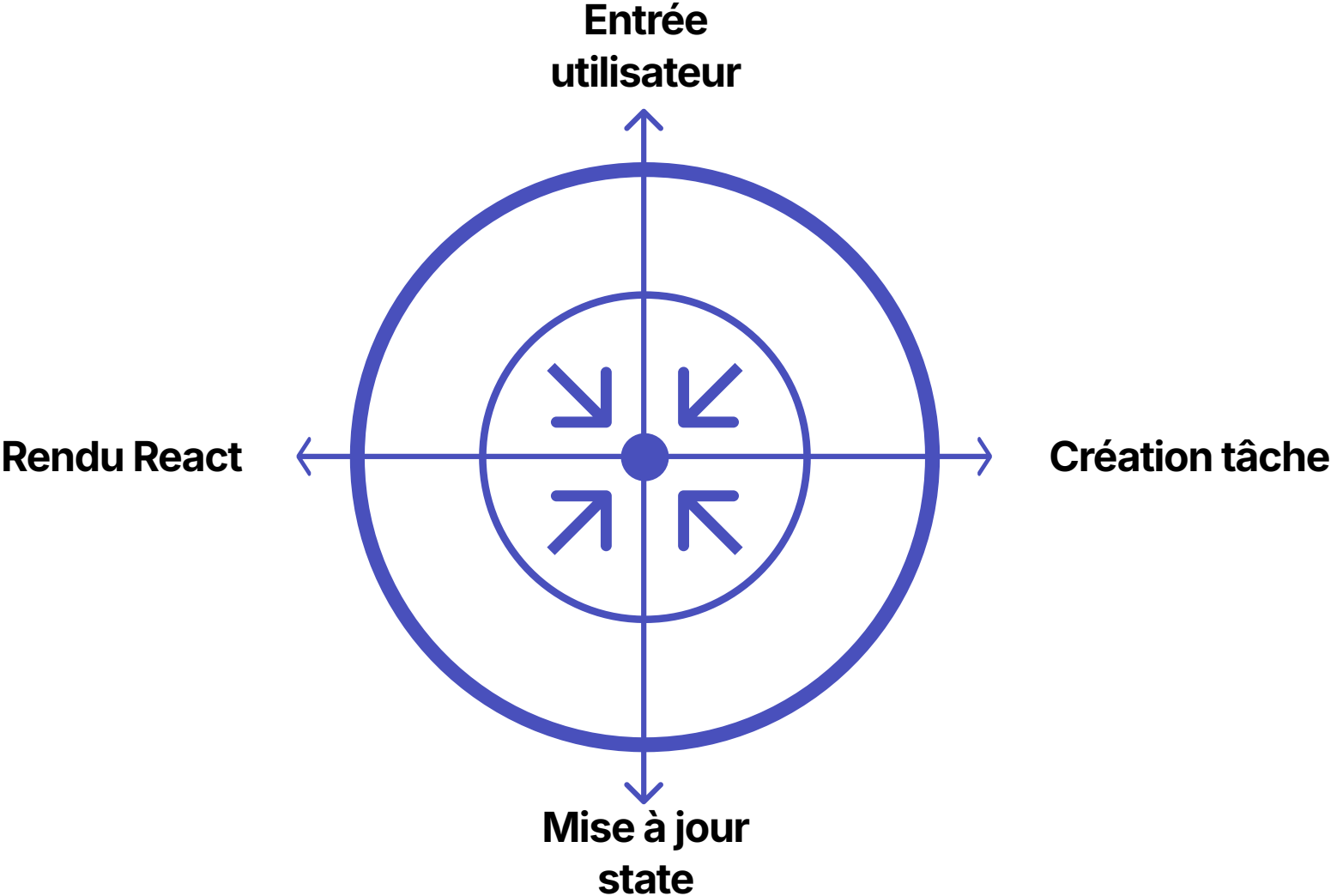
Extrait : Dashboard gère un state tasks, ajoute une tâche et bascule completed. React ré-render automatiquement quand le state change.

```
import { useState } from "react";
function Dashboard() {
  const [tasks, setTasks] = useState([]);
  const [newTask, setNewTask] = useState("");
  function addTask() { ... }
  function toggleTask(id) { ... }
  return ( ... );
}
export default Dashboard;
```

📄 Explication concise : useState conserve l'état local ; setTasks déclenche un rendu réactif.



Flux logique : ajout d'une tâche



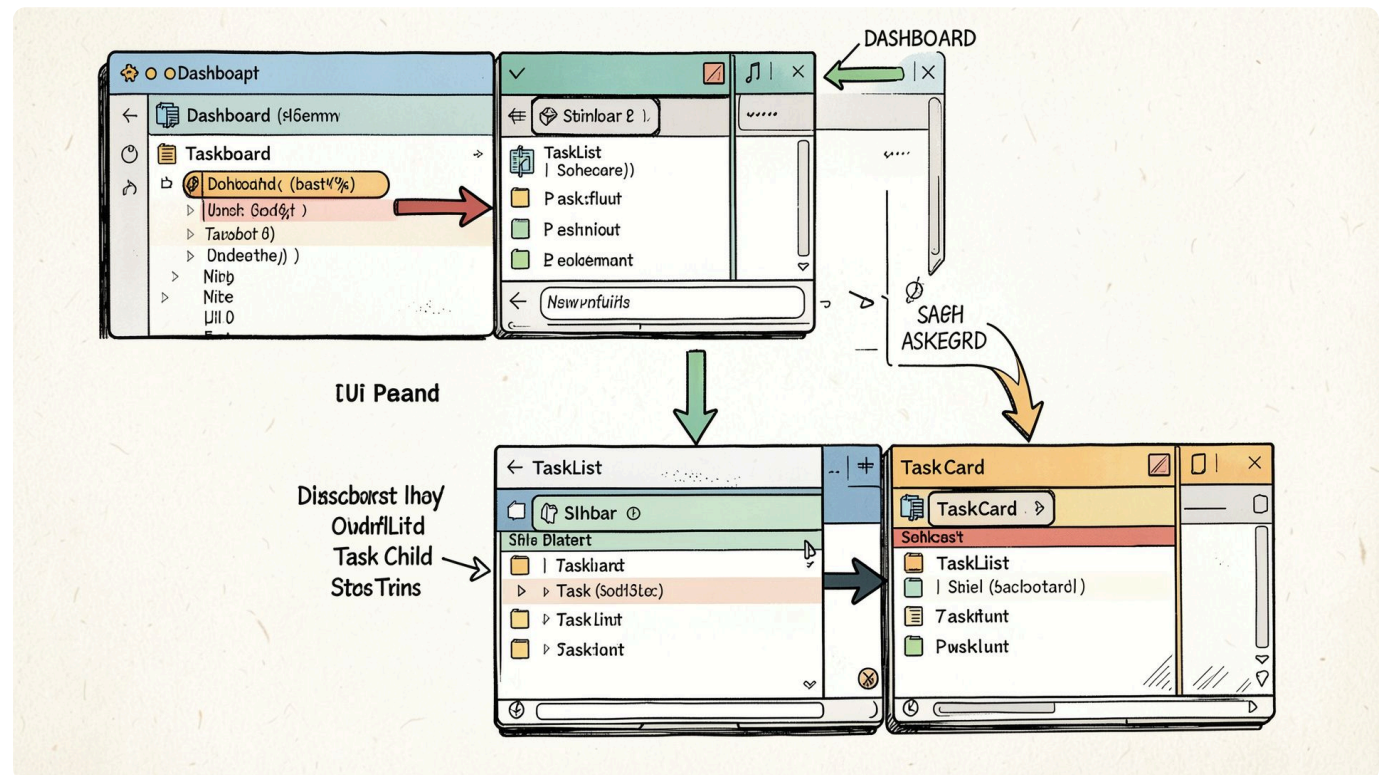
React utilise le "state" du composant pour garder les données d'affichage et déclencher le rendu quand on le modifie.

TaskCard — séparation des responsabilités

```
function TaskCard({ task, onToggle }) {
  return (
    <div className="bg-white p-4
rounded-xl shadow flex items-center
gap-3">
      <input type="checkbox" checked=
{task.completed} onChange={() =>
onToggle(task.id)} />
      <div className="flex-1">
        <h3 className="font-semibold">
{task.title}</h3>
        <p className="text-sm text-
gray-500">Priorité : {task.priority}</p>
      </div>
    </div>
  );
}
```

export default TaskCard;

Organisation : Dashboard → TaskList → TaskCard. Chaque composant a une responsabilité claire (affichage, logique, interaction).



Communication Frontend ↔ Backend

Exemple : envoyer une tâche au backend pour persistage.

```
async function addTask(title) {  
  const response = await fetch("/api/tasks", {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({ title, priority: "medium" })  
  });  
  const task = await response.json();  
  return task;  
}
```

- 1. Frontend prépare la donnée
- 2. POST /api/tasks
- 3. Backend vérifie, INSERT en PostgreSQL
- 4. Backend renvoie la tâche persistée

Backend, base de données & intelligence

Backend : routes → controllers → services → DB. Exemple d'arborescence :

```
backend/  
├── routes/  
├── controllers/  
├── services/  
├── middleware/  
└── server.js
```

Schéma des tables principales : USER, TASK, TIME_SESSION, GOAL → relations : user → tasks → time_sessions.

- ❏ Partie intelligente : l'analyse repose sur les données collectées (sessions + historique + objectifs). Ce n'est pas un diagnostic — ce sont des suggestions basées sur usage.